

SECTION 01 · LINUX

Linux OS & CLI Mastery

Senior lecture materials with commands, hardening, and Odoo server habits

AGENDA

What this section teaches

Navigate

Filesystem fluency and safe file operations

Operate

Users, processes, systemd, logs, packages

Secure

SSH, UFW, permissions, production layout

Automate

cron, env, troubleshooting checklists

18 lessons · explanations · examples · labs

METHOD

How to teach and learn this deck

- Each lesson starts with objectives, then teaching narrative
- Code/command slides are meant to be typed live or in labs
- Pitfall slides capture production failure modes
- Use the matching documentation part for full prose depth

Facilitation: These are lecture materials—pause for learner typing time after every example.

LESSONS L01–L05

Linux OS & CLI Mastery

Teaching block 1

L01 · OBJECTIVES

OS Fundamentals and Kernel Basics

- Explain the relationship between hardware, kernel, system libraries, and user processes.
- Identify where the shell, services, and Odoo workers fit in the operating system model.
- Use basic inspection commands to confirm kernel and distribution details.
- Describe why kernel resources affect application performance.

Lab target: On a lab VM, record kernel, distribution, CPU, memory, and top-level process information. Write a short note explaining which layer would own a network failure, a Python import failure, and an out-of-memory crash.

L01 · TEACHING 1

OS Fundamentals and Kernel Basics

OS Fundamentals and Kernel Basics matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands the kernel boundary and the role of user space can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to kernel version, distribution release, CPU architecture, memory, and process ownership and keeping the command output as evidence.

L01 · TEACHING 2

OS Fundamentals and Kernel Basics (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is debugging symptoms at the application layer while ignoring resource or kernel limits, which is harder to diagnose after several unrelated changes.

L01 · TEACHING 3

OS Fundamentals and Kernel Basics (continued)

In an Odoo deployment, Odoo workers, PostgreSQL, Nginx, and cron all compete for CPU time, memory, file descriptors, and network sockets connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L01 · TEACHING 4

OS Fundamentals and Kernel Basics (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Start investigations by naming the layer you are testing: hardware, kernel, service manager, process, or application. turns Linux from a source of surprises into a stable operating base for Odoo work.

L01 · KEY POINTS

What to remember

- The kernel manages CPU scheduling, memory, devices, filesystems, and network stacks.
- User space contains shells, package managers, systemd, Python, PostgreSQL, Nginx, and Odoo.
- A distribution combines the Linux kernel with tools, defaults, repositories, and service policies.
- Applications fail when kernel limits are exhausted even if their own code is correct.
- Version and architecture details matter when installing drivers, Python wheels, and database packages.
- Good incident notes separate facts observed from assumptions about root cause.

L01 · EXAMPLE 1

Inspect the operating system baseline

Inspect the operating system baseline

These commands establish the kernel, distribution, CPU, and memory baseline before troubleshooting. Capture this information in support tickets because package names, service defaults, and binary compatibility vary by distribution and version.

```
uname -a  
cat /etc/os-release  
lscpu | sed -n '1,8p'  
free -h
```

L01 · EXAMPLE 2

See user-space processes

See user-space processes

The process tree shows that every running program is a child of an existing process and has a user identity. For Odoo, this makes it clear whether work is running under systemd, an interactive shell, or an accidental manual start.

```
ps -eo pid,ppid,user,stat,comm --sort=pid | head -20  
pstree -ap | head -30
```

L01 · PITFALLS

Common mistakes

- Treating Linux as a black box and jumping directly into Odoo configuration.
- Assuming two Ubuntu releases behave identically because the commands look similar.
- Ignoring kernel limits such as open files, memory pressure, and socket backlog.
- Changing production settings without first recording the current system state.

Remember: Validate fixes in a disposable database before production.

L01 · EXTRA

Linux layers

Layer	Examples	Why Odoo developers care
Hardware	CPU, RAM, disk, NIC	Defines capacity and failure boundaries
Kernel	scheduler, memory, TCP, filesystems	Controls resources used by workers
User space	systemd, Python, PostgreSQL, Nginx	Runs the actual Odoo stack

L02 · OBJECTIVES

Terminal Navigation with pwd, cd, ls, and tree

- Navigate absolute and relative paths confidently.
- Use directory listings to understand an unfamiliar server.
- Distinguish the current shell directory from service working directories.
- Build safe habits before editing production files.

Lab target: Create a small directory tree under /tmp/odoo-navigation-lab, navigate it using absolute and relative paths, then produce a one-screen map with tree -L 3 and annotations for config, logs, and addons.

L02 · TEACHING 1

Terminal Navigation with pwd, cd, ls, and tree

Terminal Navigation with pwd, cd, ls, and tree matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem navigation and situational awareness can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to current directory, hidden files, symlinks, ownership, and directory depth and keeping the command output as evidence.

L02 · TEACHING 2

Terminal Navigation with pwd, cd, ls, and tree (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is editing or deleting the wrong file because the shell was in an unexpected directory, which is harder to diagnose after several unrelated changes.

L02 · TEACHING 3

Terminal Navigation with pwd, cd, ls, and tree (continued)

In an Odoo deployment, Odoo source, custom addons, logs, configuration, and backups usually live in different directories with different owners connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L02 · TEACHING 4

Terminal Navigation with pwd, cd, ls, and tree (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Say the path out loud, print it with pwd, and list it before running a destructive command. turns Linux from a source of surprises into a stable operating base for Odoo work.

L02 · KEY POINTS

What to remember

- `pwd` prints the shell's current working directory; it does not tell you where a service runs.
- `cd` changes directory for the current shell session only.
- `ls -la` shows hidden files, permissions, owners, sizes, and timestamps.
- `tree` is useful for understanding project shape but should be limited with `-L` on large directories.
- Absolute paths start at `/`, while relative paths start from the current directory.
- Symlinks are common in deployments and should be inspected before editing through them.

L02 · EXAMPLE 1

Build a safe navigation habit

Build a safe navigation habit

This sequence confirms the starting point, moves to the expected application directory, and inspects contents before making changes. The limited tree depth gives a useful overview without flooding the terminal on a large addons directory.

```
pwd
cd /opt/odoo
pwd
ls -la
tree -L 2 /opt/odoo
```

L02 · EXAMPLE 2

Compare absolute and relative paths

Compare absolute and relative paths

The same file can be referenced through different path forms, but only the absolute form is independent of the current directory. Scripts and service files should prefer absolute paths so they behave consistently.

```
cd /etc/odoo
ls -l odoo.conf
ls -l /etc/odoo/odoo.conf
cd /
ls -l etc/odoo/odoo.conf
```

L02 · PITFALLS

Common mistakes

- Running `rm`, `mv`, or `chmod` before checking `pwd`.
- Forgetting that hidden dotfiles affect shell and application behavior.
- Using `tree` without a depth limit on large production directories.
- Editing a symlink target without realizing where the real file lives.

Remember: Validate fixes in a disposable database before production.

L03 · OBJECTIVES

File Manipulation: Create, Copy, Move, Link, and Remove

- Create and inspect files safely from the terminal.
- Copy and move files while preserving important metadata when needed.
- Understand hard links, symbolic links, and backup naming.
- Use cautious deletion practices on servers.

Lab target: Create a fake /tmp/odoo-file-lab config file, back it up with a timestamp, edit one setting, compare with diff, then replace a current symlink to point at a new fake release directory.

L03 · TEACHING 1

File Manipulation: Create, Copy, Move, Link, and Remove

File Manipulation: Create, Copy, Move, Link, and Remove matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands safe manipulation of files and directories can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to destination paths, overwrite behavior, timestamps, ownership, and symbolic links and keeping the command output as evidence.

L03 · TEACHING 2

File Manipulation: Create, Copy, Move, Link, and Remove (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is destroying the only known-good configuration during a rushed fix, which is harder to diagnose after several unrelated changes.

L03 · TEACHING 3

File Manipulation: Create, Copy, Move, Link, and Remove (continued)

In an Odoo deployment, Odoo configuration files, custom modules, attachment filestore data, and backup archives require different handling connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L03 · TEACHING 4

File Manipulation: Create, Copy, Move, Link, and Remove (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Make reversible changes: copy first, edit second, verify third, and delete only after the replacement is proven. turns Linux from a source of surprises into a stable operating base for Odoo work.

L03 · KEY POINTS

What to remember

- touch creates an empty file or updates timestamps.
- cp copies data; cp -a preserves mode, owner, timestamps, and symlinks where permissions allow.
- mv renames within a filesystem and moves across filesystems by copying then removing.
- ln -s creates a symbolic link, which stores a path reference rather than file content.
- rm is permanent at the shell level; production cleanup should be explicit and reviewed.
- Backups need names that show what changed, when, and why.

L03 · EXAMPLE 1

Back up and edit a configuration file

Back up and edit a configuration file

The archive copy preserves the original metadata and embeds a timestamp in the backup name. A diff after editing keeps the review focused on exactly what changed before the service is restarted.

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.bak.$(date +%Y%m%d-%H%M%S)
sudo nano /etc/odoo/odoo.conf
sudo diff -u /etc/odoo/odoo.conf.bak.* /etc/odoo/odoo.conf | tail -40
```

L03 · EXAMPLE 2

Use a symlink for active custom addons

Use a symlink for active custom addons

A stable symlink such as current lets service configuration point at one path while releases change underneath it. The -n and -f flags replace the link itself rather than following an existing link as a directory.

```
sudo mkdir -p /opt/odoo/custom-addons/releases/2026-07-24
sudo ln -sf /opt/odoo/custom-addons/releases/2026-07-24 /opt/odoo/custom-addons/current
ls -l /opt/odoo/custom-addons/current
```

L03 · PITFALLS

Common mistakes

- Copying config files without preserving permissions, then wondering why services cannot read them.
- Using wildcard deletes without first listing what the wildcard expands to.
- Confusing a symlink with a physical copy of a directory.
- Leaving backup files in locations where applications accidentally load them.

Remember: Validate fixes in a disposable database before production.

L04 · OBJECTIVES

Permissions with chmod, chown, and chgrp

- Read Unix permission bits and ownership from ls output.
- Apply chmod using symbolic and numeric modes.
- Change file owners and groups safely with chown and chgrp.
- Choose permissions suitable for Odoo configuration, source code, and logs.

Lab target: In /tmp/odoo-permission-lab, create a config file and addon directory. Set ownership and permissions so a simulated odoo user can read them but an unrelated user cannot modify them.

L04 · TEACHING 1

Permissions with chmod, chown, and chgrp

Permissions with chmod, chown, and chgrp matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands ownership and permission enforcement can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to read, write, execute bits, owner, group, others, directories, and recursive changes and keeping the command output as evidence.

L04 · TEACHING 2

Permissions with chmod, chown, and chgrp (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is masking an ownership problem by making sensitive files world-writable, which is harder to diagnose after several unrelated changes.

L04 · TEACHING 3

Permissions with chmod, chown, and chgrp (continued)

In an Odoo deployment, Odoo should run as a dedicated user that can read its code and config but does not own everything on the server connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L04 · TEACHING 4

Permissions with chmod, chown, and chgrp (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Prefer the smallest permission change that makes the intended actor able to do the intended action. turns Linux from a source of surprises into a stable operating base for Odoo work.

L04 · KEY POINTS

What to remember

- Permission triplets apply to owner, group, and others in that order.
- Execute on a directory means the user can traverse it; read means the user can list names.
- `chmod 640` is common for private configuration files readable by owner and group only.
- `chown` changes owner and optionally group; `chgrp` changes only the group.
- Recursive `chmod` and `chown` are high-risk operations on production systems.
- The service user, not the admin's interactive user, must have the required access.

L04 · EXAMPLE 1

Set safe Odoo configuration permissions

Set safe Odoo configuration permissions

The root user owns the file so the service cannot silently rewrite it, while the odoo group can read it. Testing as the odoo user is more reliable than assuming permissions are correct from an admin shell.

```
sudo chown root:odoo /etc/odoo/odoo.conf
sudo chmod 640 /etc/odoo/odoo.conf
ls -l /etc/odoo/odoo.conf
sudo -u odoo test -r /etc/odoo/odoo.conf && echo "odoo can read config"
```

L04 · EXAMPLE 2

Repair ownership of a custom addon

Repair ownership of a custom addon

Directories and files often need different execute bits, so a single recursive chmod is rarely ideal. This pattern keeps source readable by the service while avoiding broad write access.

```
sudo chown -R odoo:odoo /opt/odoo/custom-addons/my_module
find /opt/odoo/custom-addons/my_module -type d -exec chmod 750 {} \;
find /opt/odoo/custom-addons/my_module -type f -exec chmod 640 {} \;
```


L04 · PITFALLS

Common mistakes

- Using `chmod 777` as a troubleshooting shortcut and leaving it in place.
- Forgetting execute permission on parent directories.
- Changing ownership of system directories recursively from the wrong path.
- Testing access as root instead of the service user.

Remember: Validate fixes in a disposable database before production.

L04 · EXTRA

Production rule

Production rule: If the fix requires `chmod 777`, the real problem has not been understood yet.

L05 · OBJECTIVES

Users, Groups, and sudo

- Explain why services should run under dedicated users.
- Inspect users, groups, and sudo privileges.
- Use sudo intentionally without hiding ownership problems.
- Separate human admin access from application runtime access.

Lab target: Create a lab service user, place it in a project group, and demonstrate a file that the group can read but cannot write. Then run a Python version check as that service user.

L05 · TEACHING 1

Users, Groups, and sudo

Users, Groups, and sudo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands identity and privilege separation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to user IDs, group memberships, service accounts, sudo rules, and command auditing and keeping the command output as evidence.

L05 · TEACHING 2

Users, Groups, and sudo (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running application processes as root and widening the blast radius of a compromise, which is harder to diagnose after several unrelated changes.

L05 · TEACHING 3

Users, Groups, and sudo (continued)

In an Odoo deployment, Odoo service users need predictable file access while administrators need controlled elevation for maintenance connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L05 · TEACHING 4

Users, Groups, and sudo (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Use sudo to perform administrative tasks, not to make normal application operations depend on root. turns Linux from a source of surprises into a stable operating base for Odoo work.

L05 · KEY POINTS

What to remember

- Each process runs with a user ID and group IDs that determine access checks.
- A service account should own only the files it must manage.
- sudo records administrative intent and can restrict which users may run privileged commands.
- Group membership is a clean way to grant shared read or write access.
- Human users should log in individually instead of sharing one admin password.
- Root-owned runtime files often reveal that a command was run with the wrong identity.

L05 · EXAMPLE 1

Inspect identity and group membership

Inspect identity and group membership

These commands answer who you are, who the service user is, and what privileged commands are allowed. The output is essential when a file is readable in your shell but not by the Odoo service.

```
id
id odoo
getent passwd odoo
getent group odoo
sudo -l
```

L05 · EXAMPLE 2

Run a check as the service user

Run a check as the service user

Running a command as the service user reproduces the environment and access rules that systemd will use. The -H flag sets HOME consistently so user-specific configuration does not leak from the admin account.

```
sudo -u odoo -H bash -lc 'whoami; pwd; python3 --version'  
sudo -u odoo -H test -x /opt/odoo/venv/bin/python && echo "venv executable"
```

L05 · PITFALLS

Common mistakes

- Running Odoo manually with sudo and creating root-owned cache files.
- Sharing one SSH account among administrators.
- Adding broad passwordless sudo rules without command boundaries.
- Changing service file ownership instead of fixing group membership.

Remember: Validate fixes in a disposable database before production.

LESSONS L06–L10

Linux OS & CLI Mastery

Teaching block 2

L06 · OBJECTIVES

Processes with top, htop, ps, and kill

- List processes and interpret PID, parent PID, state, CPU, and memory columns.
- Use top or htop to observe live server pressure.
- Terminate processes with the least disruptive signal first.
- Connect process behavior to Odoo worker and cron performance.

Lab target: Start a harmless sleep process, find it with ps and pgrep, send TERM, confirm it exited, then repeat with a Python process that prints its PID and handles KeyboardInterrupt.

L06 · TEACHING 1

Processes with top, htop, ps, and kill

Processes with top, htop, ps, and kill matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands runtime process observation and control can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to PIDs, process states, parent-child relationships, CPU use, RSS memory, and signals and keeping the command output as evidence.

L06 · TEACHING 2

Processes with top, htop, ps, and kill (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is killing the wrong worker or masking the underlying cause of resource exhaustion, which is harder to diagnose after several unrelated changes.

L06 · TEACHING 3

Processes with top, htop, ps, and kill (continued)

In an Odoo deployment, Odoo may run multiple HTTP workers, longpolling workers, cron workers, PostgreSQL sessions, and background jobs connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L06 · TEACHING 4

Processes with top, htop, ps, and kill (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A kill command is a last step in an investigation, not the investigation itself. turns Linux from a source of surprises into a stable operating base for Odoo work.

L06 · KEY POINTS

What to remember

- ps gives a point-in-time process list; top and htop show live changes.
- PID identifies a process instance, while command names may repeat many times.
- SIGTERM asks a process to stop cleanly; SIGKILL stops it immediately and prevents cleanup.
- High CPU, high memory, and uninterruptible IO wait suggest different root causes.
- Parent PID helps distinguish systemd-managed services from manual shells.
- Repeatedly killing processes without logs creates unstable systems and lost evidence.

L06 · EXAMPLE 1

Find Odoo-related processes

Find Odoo-related processes

Sorting by memory and filtering by the odoo user quickly reveals whether application workers dominate the server. Elapsed time helps separate a fresh runaway process from a long-running service.

```
ps -eo pid,ppid,user,stat,%cpu,%mem,etime,cmd --sort=-%mem | head -20  
pgrep -a -u odoo python  
top -o %MEM
```

L06 · EXAMPLE 2

Signal a process deliberately

Signal a process deliberately

TERM gives the process a chance to close files and database connections cleanly. KILL should be explicit and documented because it can interrupt transactions or leave temporary files behind.

```
sudo kill -TERM 12345
sleep 3
ps -p 12345 -o pid,stat,cmd
sudo kill -KILL 12345 # only if the process ignored TERM and impact is understood
```

L06 · PITFALLS

Common mistakes

- Using kill -9 as the first response.
- Matching processes by a vague name and terminating unrelated commands.
- Ignoring parent processes that will immediately restart the child.
- Confusing memory percent with actual RSS on large-memory servers.

Remember: Validate fixes in a disposable database before production.

L07 · OBJECTIVES

systemd Services

- Explain systemd units, service states, and restart policies.
- Inspect and manage Odoo service lifecycle with systemctl.
- Understand unit files, drop-ins, and daemon reloads.
- Use systemd status output as a first troubleshooting artifact.

Lab target: Inspect a sample odoo.service unit, identify ExecStart, User, Group, WorkingDirectory, and EnvironmentFile, then create a hypothetical drop-in that changes only the restart policy.

L07 · TEACHING 1

systemd Services

systemd Services matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands service supervision and repeatable startup can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit names, active state, main PID, ExecStart, environment, dependencies, and restart behavior and keeping the command output as evidence.

L07 · TEACHING 2

systemd Services (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is starting Odoo manually and leaving production outside the service manager, which is harder to diagnose after several unrelated changes.

L07 · TEACHING 3

systemd Services (continued)

In an Odoo deployment, systemd keeps Odoo, PostgreSQL, and Nginx predictable across reboots and failures connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L07 · TEACHING 4

systemd Services (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. If systemd is responsible for the service, always ask systemd what it thinks before changing files. turns Linux from a source of surprises into a stable operating base for Odoo work.

L07 · KEY POINTS

What to remember

- A service unit describes how a daemon starts, stops, reloads, and restarts.
- `systemctl status` combines state, main PID, recent logs, and exit code context.
- After editing unit files or drop-ins, run `systemctl daemon-reload`.
- Restart policies should match the failure mode and avoid tight crash loops.
- `EnvironmentFile` and `WorkingDirectory` affect how Python and Odoo locate resources.
- Manual starts should be used only for controlled diagnostics and then stopped.

L07 · EXAMPLE 1

Inspect and restart an Odoo service

Inspect and restart an Odoo service

status answers whether the service is active and shows the immediate failure context if it is not. cat displays the effective unit content, including drop-ins, which prevents debugging the wrong file.

```
sudo systemctl status odoo --no-pager
sudo systemctl cat odoo
sudo systemctl restart odoo
sudo systemctl is-active odoo
```

L07 · EXAMPLE 2

Create a safe override directory

Create a safe override directory

Drop-ins preserve vendor or deployment unit files while making local changes explicit. daemon-reload is required because systemd caches unit definitions.

```
sudo systemctl edit odoo
# Add only the changed settings in the editor, for example:
# [Service]
# Restart=on-failure
sudo systemctl daemon-reload
sudo systemctl restart odoo
```

L07 · PITFALLS

Common mistakes

- Editing a unit file and restarting without daemon-reload.
- Debugging an old service name while the active unit has a different name.
- Running Odoo manually and assuming systemd uses the same environment.
- Setting Restart=always without understanding a crash loop.

Remember: Validate fixes in a disposable database before production.

L08 · OBJECTIVES

journalctl and Log Files

- Read systemd journal entries for a service.
- Filter logs by unit, time range, priority, and boot.
- Combine journal output with application log files.
- Preserve useful evidence during Odoo incidents.

Lab target: Restart a harmless sample service or timer, collect journal entries for the last 10 minutes, identify the first and last relevant lines, and write a five-line incident timeline.

L08 · TEACHING 1

journalctl and Log Files

journalctl and Log Files matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands log-based investigation can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to unit filters, timestamps, boot IDs, priority levels, application log paths, and rotation and keeping the command output as evidence.

L08 · TEACHING 2

journalctl and Log Files (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is losing the first failure message by only looking at the last repeated error, which is harder to diagnose after several unrelated changes.

L08 · TEACHING 3

journalctl and Log Files (continued)

In an Odoo deployment, Odoo logs often show Python tracebacks while systemd logs show process exit, restart, and environment failures connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L08 · TEACHING 4

journalctl and Log Files (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Start with the first error in the relevant time window, then follow the chain forward. turns Linux from a source of surprises into a stable operating base for Odoo work.

L08 · KEY POINTS

What to remember

- `journalctl -u` filters logs for a systemd unit.
- The `--since` and `--until` options keep searches tied to the incident window.
- The current boot can be selected with `-b`, and previous boots with `-b -1`.
- Application logs may live in `/var/log/odoo` or another configured path.
- Tracebacks should be read from top context to final exception, not just the last line.
- Log rotation can move older evidence into compressed files.

L08 · EXAMPLE 1

Read recent service logs

Read recent service logs

The first command establishes recent context, the second focuses on warnings and errors, and the third stays within the current boot. Time filters prevent unrelated old failures from confusing the incident timeline.

```
sudo journalctl -u odoo --since "30 minutes ago" --no-pager  
sudo journalctl -u odoo -p warning..alert --since today --no-pager  
sudo journalctl -u odoo -b --no-pager | tail -80
```

L08 · EXAMPLE 2

Follow application logs during restart

Follow application logs during restart

Following the application log during a restart shows startup module loading, database connection failures, and Python exceptions as they happen. Use this with `journalctl status` so you see both Odoo's view and systemd's view.

```
sudo tail -F /var/log/odoo/odoo.log  
# In another terminal:  
sudo systemctl restart odoo
```

L08 · PITFALLS

Common mistakes

- Searching logs without a time window.
- Reading only the final traceback line and missing the real cause above it.
- Ignoring systemd restart messages when the application log is empty.
- Overwriting logs during cleanup before preserving the incident evidence.

Remember: Validate fixes in a disposable database before production.

L09 · OBJECTIVES

Editing Files with nano and vim

- Choose nano or vim based on task complexity and operator comfort.
- Open, edit, save, and exit safely in both editors.
- Avoid accidental configuration damage while editing as root.
- Validate edits before restarting services.

Lab target: Edit a copied sample config with both nano and vim. Change one value, save it, show the diff, then intentionally abandon a second change without saving.

L09 · TEACHING 1

Editing Files with nano and vim

Editing Files with nano and vim matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands terminal editing discipline can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to editor modes, save behavior, sudo usage, temporary files, syntax visibility, and post-edit validation and keeping the command output as evidence.

L09 · TEACHING 2

Editing Files with nano and vim (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making an invisible typo in a production configuration file, which is harder to diagnose after several unrelated changes.

L09 · TEACHING 3

Editing Files with nano and vim (continued)

In an Odoo deployment, Odoo and Nginx configuration changes often need exact key names, indentation, and path values connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L09 · TEACHING 4

Editing Files with nano and vim (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. The best editor is the one you can exit reliably during an incident. turns Linux from a source of surprises into a stable operating base for Odoo work.

L09 · KEY POINTS

What to remember

- nano is approachable for small edits and displays common shortcuts.
- vim is modal, powerful, and widely available on servers.
- Always back up important files before editing them as root.
- Use validation commands before restarting services that depend on edited config.
- sudoedit can be safer than running a full editor process as root.
- Editor swap and backup files should not be left where applications will load them.

L09 · EXAMPLE 1

Safe edit sequence

Safe edit sequence

sudoedit copies the file to a temporary location, opens it with your editor, and writes it back with elevated permissions. The diff provides a review checkpoint before the changed service is restarted.

```
sudo cp -a /etc/odoo/odoo.conf /etc/odoo/odoo.conf.pre-edit  
sudoedit /etc/odoo/odoo.conf  
sudo diff -u /etc/odoo/odoo.conf.pre-edit /etc/odoo/odoo.conf
```

L09 · EXAMPLE 2

Essential vim commands

Essential vim commands

Most vim emergencies come from not recognizing insert mode versus normal mode. Knowing these commands is enough to make a controlled edit or escape safely.

```
vim /tmp/example.conf
# i          enter insert mode
# Esc       return to normal mode
# :w        write file
# :q        quit
# :wq       write and quit
# :q!       quit without saving
```

L09 · PITFALLS

Common mistakes

- Opening a root editor without a backup.
- Saving editor temporary files in a directory that is scanned by Odoo.
- Restarting services before checking a diff or validation command.
- Panicking in vim instead of pressing Esc and using :q! to exit without saving.

Remember: Validate fixes in a disposable database before production.

L10 · OBJECTIVES

Networking with ip, ping, and ss

- Inspect IP addresses, routes, and listening sockets.
- Use ping for basic reachability without overinterpreting it.
- Use ss to confirm which process owns a port.
- Troubleshoot Odoo, PostgreSQL, and reverse proxy connectivity.

Lab target: Draw a simple network path from browser to Nginx to Odoo to PostgreSQL. On a lab host, collect ip addr, ip route, and ss output and label which ports should be public.

L10 · TEACHING 1

Networking with ip, ping, and ss

Networking with ip, ping, and ss matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands network observation from the host can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to interfaces, addresses, routes, DNS assumptions, TCP ports, process ownership, and listening bind addresses and keeping the command output as evidence.

L10 · TEACHING 2

Networking with ip, ping, and ss (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is assuming the application is broken when the port is not listening or traffic is routed elsewhere, which is harder to diagnose after several unrelated changes.

L10 · TEACHING 3

Networking with ip, ping, and ss (continued)

In an Odoo deployment, Odoo typically listens on localhost or a private interface while Nginx exposes HTTP and HTTPS to users connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L10 · TEACHING 4

Networking with ip, ping, and ss (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Separate reachability, name resolution, listening state, firewall policy, and application response. turns Linux from a source of surprises into a stable operating base for Odoo work.

L10 · KEY POINTS

What to remember

- `ip addr` shows interface addresses; `ip route` shows where packets will go.
- `ping` tests ICMP reachability, not whether TCP application ports are open.
- `ss -ltnp` shows listening TCP sockets and owning processes.
- A service bound to `127.0.0.1` is reachable locally but not from another host.
- PostgreSQL and Odoo ports should usually be private, with Nginx handling public HTTP.
- Network troubleshooting improves when each layer is tested independently.

L10 · EXAMPLE 1

Inspect addresses, routes, and ports

Inspect addresses, routes, and ports

The commands move from host addressing to routing, reachability, and listening sockets. For Odoo, the socket list often reveals whether the service is bound only to localhost or exposed more broadly.

```
ip addr show
ip route show
ping -c 3 8.8.8.8
ss -ltnp | grep -E ':(8069|8072|80|443|5432)\b'
```

L10 · EXAMPLE 2

Test an Odoo port locally

Test an Odoo port locally

A local HTTP check proves the application responds before investigating DNS, load balancers, or public firewalls. The ss filter confirms which process owns the port.

```
curl -I http://127.0.0.1:8069  
curl -I http://localhost:8069  
ss -ltnp 'sport = :8069'
```


L10 · PITFALLS

Common mistakes

- Treating a successful ping as proof that HTTP or PostgreSQL works.
- Forgetting that localhost from one container or VM is not another host's localhost.
- Leaving Odoo bound to a public interface unnecessarily.
- Ignoring DNS and testing only raw IP addresses.

Remember: Validate fixes in a disposable database before production.

LESSONS L11–L15

Linux OS & CLI Mastery

Teaching block 3

L11 · OBJECTIVES

SSH Keys and Hardening

- Explain public-key SSH authentication.
- Generate and install SSH keys safely.
- Apply basic sshd hardening for production servers.
- Avoid locking yourself out during access changes.

Lab target: On a disposable VM, create a new SSH key, install it for a non-root user, test login, then draft a hardened sshd_config snippet with comments explaining each setting.

L11 · TEACHING 1

SSH Keys and Hardening

SSH Keys and Hardening matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands secure remote administration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to key pairs, `authorized_keys`, `sshd_config`, password login, root login, ports, and active sessions and keeping the command output as evidence.

L11 · TEACHING 2

SSH Keys and Hardening (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is weakening or breaking the only administrative access path to a production host, which is harder to diagnose after several unrelated changes.

L11 · TEACHING 3

SSH Keys and Hardening (continued)

In an Odoo deployment, Odoo servers need secure admin access because shell access often implies database, file, and secret access connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L11 · TEACHING 4

SSH Keys and Hardening (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Make SSH changes with a second session open and a rollback path ready. turns Linux from a source of surprises into a stable operating base for Odoo work.

L11 · KEY POINTS

What to remember

- The private key stays on the client; the public key is placed on the server.
- `authorized_keys` controls which public keys can log in as a user.
- `PasswordAuthentication no` reduces brute-force risk after keys are confirmed.
- `PermitRootLogin no` encourages named users and sudo auditability.
- `sshd` configuration should be tested before reload or restart.
- Firewall and cloud security group rules must match the chosen SSH port.

L11 · EXAMPLE 1

Create and install a key

Create and install a key

Ed25519 keys are compact and strong for modern SSH clients. The final command proves the key works before password login is disabled.

```
ssh-keygen -t ed25519 -C "admin@example.com"  
ssh-copy-id admin@odoo-server.example.com  
ssh admin@odoo-server.example.com 'whoami; hostname'
```

L11 · EXAMPLE 2

Test sshd configuration before reload

Test sshd configuration before reload

sshd -t validates syntax without applying the configuration. Reload keeps existing sessions alive, which is safer than a restart while verifying access.

```
sudo cp -a /etc/ssh/sshd_config /etc/ssh/sshd_config.pre-hardening
sudoedit /etc/ssh/sshd_config
sudo sshd -t
sudo systemctl reload ssh
```

L11 · PITFALLS

Common mistakes

- Disabling password login before confirming key login works.
- Closing the only active session immediately after changing `sshd_config`.
- Sharing one private key across administrators.
- Changing the SSH port without updating firewall or cloud rules.

Remember: Validate fixes in a disposable database before production.

L12 · OBJECTIVES

Environment Variables and bashrc

- Read and set environment variables in a shell.
- Distinguish shell startup files from systemd service environments.
- Use PATH, HOME, and application variables intentionally.
- Avoid hiding production settings in interactive-only files.

Lab target: Create a shell variable and an exported variable, run a child Python command that prints `os.environ`, and explain why only one value is visible. Then draft a service EnvironmentFile for a sample Odoo variable.

L12 · TEACHING 1

Environment Variables and bashrc

Environment Variables and bashrc matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands process environment configuration can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to exported variables, shell startup order, PATH lookup, virtual environments, and systemd EnvironmentFile usage and keeping the command output as evidence.

L12 · TEACHING 2

Environment Variables and bashrc (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is testing with variables from `.bashrc` that the Odoo service never receives, which is harder to diagnose after several unrelated changes.

L12 · TEACHING 3

Environment Variables and bashrc (continued)

In an Odoo deployment, Odoo configuration may depend on PATH, Python virtual environments, locale, addons paths, and database connection variables connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L12 · TEACHING 4

Environment Variables and bashrc (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. If a service needs a value, put it in the service configuration path, not only in an interactive shell profile. turns Linux from a source of surprises into a stable operating base for Odoo work.

L12 · KEY POINTS

What to remember

- Environment variables are inherited from parent process to child process.
- `export` makes a shell variable available to child processes.
- `.bashrc` affects interactive bash shells, not all services.
- `PATH` controls command lookup order and can explain different Python binaries.
- `systemd` units can use `Environment` and `EnvironmentFile` for service-specific values.
- Secrets in environment files need strict ownership and permissions.

L12 · EXAMPLE 1

Inspect shell environment

Inspect shell environment

These commands show which executable the shell will run and which inherited values are visible. A mismatch between command -v and service ExecStart is a frequent source of deployment confusion.

```
echo "$PATH"  
printenv | sort | grep -E '^(HOME|LANG|PATH|VIRTUAL_ENV)='  
command -v python3  
python3 -c 'import sys; print(sys.executable)'
```

L12 · EXAMPLE 2

Use an EnvironmentFile with systemd

Use an EnvironmentFile with systemd

An environment file makes service configuration explicit and auditable. The permissions allow the odoo group to read values while preventing broad disclosure.

```
sudo install -m 640 -o root -g odoo /dev/null /etc/odoo/odoo.env
echo 'ODOO_RC=/etc/odoo/odoo.conf' | sudo tee -a /etc/odoo/odoo.env
sudo systemctl edit odoo
# [Service]
# EnvironmentFile=/etc/odoo/odoo.env
```

L12 · PITFALLS

Common mistakes

- Adding production variables only to ~/.bashrc and expecting systemd to see them.
- Modifying PATH globally when only one service needs a specific executable.
- Storing secrets in world-readable environment files.
- Assuming sudo preserves the caller's full environment.

Remember: Validate fixes in a disposable database before production.

L13 · OBJECTIVES

Cron Jobs

- Understand cron timing syntax and execution environment.
- Create safe scheduled maintenance tasks.
- Redirect cron output for observability.
- Avoid conflicts between cron and Odoo's internal scheduler.

Lab target: Write a cron entry that runs a harmless script every five minutes, logs stdout and stderr, and uses flock. Then intentionally remove PATH from the script and explain the failure.

L13 · TEACHING 1

Cron Jobs

Cron Jobs matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands scheduled command execution can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to minute, hour, day fields, user crontabs, system crontabs, PATH, working directory, and output handling and keeping the command output as evidence.

L13 · TEACHING 2

Cron Jobs (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is running a maintenance command with a minimal cron environment and no logs, which is harder to diagnose after several unrelated changes.

L13 · TEACHING 3

Cron Jobs (continued)

In an Odoo deployment, Odoo has its own scheduled actions, but server tasks such as backups, cleanup, and certificate checks may use cron connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L13 · TEACHING 4

Cron Jobs (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Every cron job should state who runs it, what environment it expects, where output goes, and how failure is noticed. turns Linux from a source of surprises into a stable operating base for Odoo work.

L13 · KEY POINTS

What to remember

- User crontabs run as that user; `/etc/cron.d` entries include a user field.
- Cron has a small environment and often a different `PATH` from interactive shells.
- Use absolute paths for commands and files in cron jobs.
- Redirect stdout and stderr to logs or monitoring hooks.
- Locking prevents overlapping runs when a previous job is slow.
- Odoo database backups should be tested for restore, not just scheduled for creation.

L13 · EXAMPLE 1

A safe backup cron pattern

A safe backup cron pattern

This system cron entry declares its shell and PATH, runs as postgres, prevents overlap with flock, and captures output. The command still needs a tested script and restore procedure before it is considered production-ready.

```
# /etc/cron.d/odoo-backup
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

15 2 * * * postgres flock -n /var/lock/odoo-backup.lock /usr/local/bin/backup-odoo-db >> /var/log/odoo/backup.log 2>&1
```

L13 · EXAMPLE 2

Preview a cron schedule

Preview a cron schedule

Servers may use both cron and systemd timers, so list both before assuming how scheduling works. Searching cron directories identifies system-wide jobs that are not visible in a user's crontab.

```
crontab -l  
systemctl list-timers --all  
grep -R "backup-odoo" /etc/cron* 2>/dev/null
```

L13 · PITFALLS

Common mistakes

- Using relative paths in cron commands.
- Ignoring stderr so failures disappear into local mail or nowhere.
- Scheduling backups without restore tests.
- Letting slow jobs overlap and overload the database server.

Remember: Validate fixes in a disposable database before production.

L14 · OBJECTIVES

APT Package Management

- Update package metadata and install packages with apt.
- Understand repositories, package versions, and upgrades.
- Inspect installed packages and service changes.
- Apply package changes safely on Odoo servers.

Lab target: On a lab Ubuntu system, inspect the candidate version of nginx, run a simulated upgrade, install a small package, and record which services changed state.

L14 · TEACHING 1

APT Package Management

APT Package Management matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands Debian and Ubuntu package lifecycle can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to apt update, apt install, apt upgrade, repositories, package holds, service restarts, and changelog awareness and keeping the command output as evidence.

L14 · TEACHING 2

APT Package Management (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is upgrading core packages during business hours without understanding service impact, which is harder to diagnose after several unrelated changes.

L14 · TEACHING 3

APT Package Management (continued)

In an Odoo deployment, Odoo servers depend on OS packages for Python, PostgreSQL clients, Nginx, wkhtmltopdf alternatives, fonts, and build tools connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L14 · TEACHING 4

APT Package Management (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Treat package changes like deployments: review, schedule, apply, verify, and record. turns Linux from a source of surprises into a stable operating base for Odoo work.

L14 · KEY POINTS

What to remember

- apt update refreshes metadata; it does not upgrade installed packages.
- apt install changes the system and may start or restart services.
- apt-cache policy shows candidate versions and repositories.
- apt-mark hold can prevent a sensitive package from upgrading unexpectedly.
- Security updates are important but still need operational timing.
- Build dependencies should be removed or minimized on hardened production hosts.

L14 · EXAMPLE 1

Inspect and install a package

Inspect and install a package

The policy command shows where the package will come from before installation. After installing, verify both package state and the service that the package may have started.

```
sudo apt update
apt-cache policy nginx
sudo apt install nginx
dpkg -l | grep '^ii' | grep nginx
systemctl status nginx --no-pager
```

L14 · EXAMPLE 2

Review pending upgrades

Review pending upgrades

The simulated upgrade shows planned actions without changing the server. Use it before maintenance windows to identify packages that may restart services or require reboots.

```
sudo apt update  
apt list --upgradable  
sudo apt -s upgrade
```

L14 · PITFALLS

Common mistakes

- Confusing apt update with apt upgrade.
- Installing from random third-party repositories without trust or maintenance review.
- Upgrading PostgreSQL-related packages without checking database compatibility.
- Leaving build tools installed on minimal production machines unnecessarily.

Remember: Validate fixes in a disposable database before production.

L15 · OBJECTIVES

Disk and Memory Usage with df, du, and free

- Measure filesystem capacity with df.
- Find large directories and files with du.
- Interpret memory and swap information with free.
- Connect disk and memory pressure to Odoo failures.

Lab target: Fill a small lab directory with test files, use du to find the largest path, then remove only the test data. Record df output before and after and explain why the numbers changed.

L15 · TEACHING 1

Disk and Memory Usage with df, du, and free

Disk and Memory Usage with df, du, and free matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands capacity and pressure diagnosis can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to mounted filesystems, inode use, directory growth, cache memory, swap use, and database storage paths and keeping the command output as evidence.

L15 · TEACHING 2

Disk and Memory Usage with df, du, and free (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is cleaning the wrong directory while the full filesystem remains full, which is harder to diagnose after several unrelated changes.

L15 · TEACHING 3

Disk and Memory Usage with df, du, and free (continued)

In an Odoo deployment, Odoo attachments, logs, PostgreSQL data, backups, and temporary exports can grow independently connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L15 · TEACHING 4

Disk and Memory Usage with df, du, and free (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Always identify the full mount point before deleting data. turns Linux from a source of surprises into a stable operating base for Odoo work.

L15 · KEY POINTS

What to remember

- `df -h` shows filesystem capacity by mount point.
- `df -ih` shows inode usage, which can be exhausted even when space remains.
- `du -sh` summarizes directory size; `sort` helps find the largest consumers.
- `free -h` separates used memory from available memory and filesystem cache.
- Swap activity can indicate memory pressure but is not automatically bad.
- Database and filestore cleanup require application-aware retention rules.

L15 · EXAMPLE 1

Find the full mount and largest directories

Find the full mount and largest directories

The -x flag keeps du on one filesystem, which prevents chasing data across mounted volumes. Checking inodes helps diagnose failures caused by too many small files, such as excessive sessions or attachments.

```
df -h
df -ih
sudo du -xhd1 /var | sort -h
sudo du -xhd1 /opt | sort -h
```

L15 · EXAMPLE 2

Inspect memory pressure

Inspect memory pressure

free shows available memory, vmstat shows activity over time, and ps identifies the largest resident processes. Together they distinguish a busy but healthy cache from a server that is swapping heavily.

```
free -h  
vmstat 1 5  
ps -eo pid,user,%mem,rss,cmd --sort=-rss | head -15
```

L15 · PITFALLS

Common mistakes

- Deleting files from a different mount point than the one that is full.
- Ignoring inode exhaustion.
- Clearing application data without retention approval.
- Mistaking Linux filesystem cache for wasted memory.

Remember: Validate fixes in a disposable database before production.

LESSONS L16–L18

Linux OS & CLI Mastery

Teaching block 4

L16 · OBJECTIVES

UFW Firewall

- Explain host firewall allow and deny rules.
- Use UFW to expose only required services.
- Verify firewall status before and after changes.
- Design firewall policy for an Odoo server behind a reverse proxy.

Lab target: Design a UFW policy for a single-server Odoo deployment with Nginx, PostgreSQL local only, and SSH from a VPN range. Explain every allowed port.

L16 · TEACHING 1

UFW Firewall

UFW Firewall matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands host-level network policy can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to default policies, allowed ports, source restrictions, IPv4 and IPv6, ordering, and rollback and keeping the command output as evidence.

L16 · TEACHING 2

UFW Firewall (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is locking out SSH or exposing internal Odoo and PostgreSQL ports publicly, which is harder to diagnose after several unrelated changes.

L16 · TEACHING 3

UFW Firewall (continued)

In an Odoo deployment, a typical Odoo host should expose SSH and HTTPS while keeping PostgreSQL and backend Odoo ports private connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L16 · TEACHING 4

UFW Firewall (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. Before enabling a firewall, confirm the active SSH rule in the same session. turns Linux from a source of surprises into a stable operating base for Odoo work.

L16 · KEY POINTS

What to remember

- UFW is a friendly interface over Linux firewall rules.
- Default deny incoming and allow outgoing is common for application hosts.
- Rules can restrict source IPs for SSH or administrative ports.
- Nginx should usually expose 80 and 443; Odoo 8069 should stay private.
- IPv6 must be considered if the host has public IPv6 addresses.
- Firewall changes need verification from both local and remote perspectives.

L16 · EXAMPLE 1

Enable a conservative Odoo firewall

Enable a conservative Odoo firewall

This allows administration and public web traffic while denying other incoming connections. OpenSSH should be allowed before enabling UFW to avoid cutting off the active session.

```
sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
```

L16 · EXAMPLE 2

Restrict SSH to an admin network

Restrict SSH to an admin network

Source-restricted SSH reduces attack surface when administrators connect from known networks. Apply this only after confirming the actual admin source ranges and any VPN paths.

```
sudo ufw delete allow OpenSSH
sudo ufw allow from 203.0.113.0/24 to any port 22 proto tcp
sudo ufw status numbered
```


L16 · PITFALLS

Common mistakes

- Enabling UFW before allowing SSH.
- Exposing PostgreSQL or Odoo backend ports to the entire internet.
- Forgetting IPv6 rules on dual-stack hosts.
- Deleting numbered rules without rechecking numbers after each deletion.

Remember: Validate fixes in a disposable database before production.

L17 · OBJECTIVES

Production Directory Layout for Odoo

- Map common Odoo directories by responsibility.
- Separate source code, configuration, logs, data, backups, and virtual environments.
- Choose ownership and permissions for each area.
- Make deployments understandable for future maintainers.

Lab target: Design a directory layout for a production Odoo server. Include owner, group, permission mode, backup policy, and which paths are referenced by the systemd service.

L17 · TEACHING 1

Production Directory Layout for Odoo

Production Directory Layout for Odoo matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands filesystem organization for maintainable deployments can reason from first principles instead of copying commands blindly.

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to config paths, addons paths, virtualenvs, logs, filestore, backups, releases, and ownership boundaries and keeping the command output as evidence.

L17 · TEACHING 2

Production Directory Layout for Odoo (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is mixing code, data, and backups until upgrades and restores become unsafe, which is harder to diagnose after several unrelated changes.

L17 · TEACHING 3

Production Directory Layout for Odoo (continued)

In an Odoo deployment, Odoo deployments require clear paths for core source, custom addons, enterprise addons, Python environment, configuration, logs, and filestore connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L17 · TEACHING 4

Production Directory Layout for Odoo (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A directory layout is operational documentation; if it is surprising, it will fail during incidents. turns Linux from a source of surprises into a stable operating base for Odoo work.

L17 · KEY POINTS

What to remember

- `/etc/odoo` is a natural place for root-owned configuration.
- `/opt/odoo` often holds application source, releases, addons, and virtual environments.
- `/var/log/odoo` should contain logs with rotation configured.
- `/var/lib/odoo` holds runtime data such as filestore when using packaged conventions.
- Backups should be isolated from live code and tested for restore.
- Symlinked release directories make rollback easier when used carefully.

L17 · EXAMPLE 1

Example directory map

Example directory map

This layout separates configuration, source, virtual environment, runtime data, logs, and backups. The exact paths may vary, but the responsibilities should remain explicit.

```
/etc/odoo/odoo.conf  
/opt/odoo/src/odoo  
/opt/odoo/enterprise  
/opt/odoo/custom-addons/current  
/opt/odoo/venv  
/var/lib/odoo/.local/share/Odoo/filestore  
/var/log/odoo/odoo.log  
/srv/backups/odoo
```


L17 · EXAMPLE 2

Check ownership boundaries

Check ownership boundaries

namei shows permissions on each path component, which catches parent-directory traversal problems. The tests confirm the service can read configuration but cannot casually rewrite it.

```
sudo namei -l /etc/odoo/odoo.conf
sudo namei -l /opt/odoo/custom-addons/current
sudo -u odoo test -r /etc/odoo/odoo.conf && echo readable
sudo -u odoo test -w /etc/odoo/odoo.conf || echo not writable
```

L17 · PITFALLS

Common mistakes

- Storing backups inside the live addons path.
- Letting the service user own root-level configuration.
- Putting virtual environments in home directories tied to human users.
- Using undocumented paths that only one administrator understands.

Remember: Validate fixes in a disposable database before production.

L17 · EXTRA

Directory responsibilities

Path	Typical owner	Responsibility
/etc/odoo	root:odoo	Service configuration
/opt/odoo	odoo:odoo	Source, addons, and virtualenv
/var/log/odoo	odoo:adm	Application logs
/srv/backups/odoo	root:backup	Restore-tested backups

L18 · OBJECTIVES

Troubleshooting Checklist for Odoo Servers

- Apply a structured checklist during Odoo outages.
- Collect evidence before changing the server.
- Move from infrastructure checks to application checks logically.
- Write concise incident notes and next actions.

Lab target: Use the checklist against a deliberately broken lab Odoo-like service. Record observations, identify the failing layer, make one fix, verify recovery, and write a brief post-incident note.

L18 · TEACHING 1

Troubleshooting Checklist for Odoo Servers

Troubleshooting Checklist for Odoo Servers matters because Odoo servers are ordinary Linux systems before they are application servers. A developer who understands systematic production troubleshooting can reason from first principles instead of copying commands blindly.

L18 · TEACHING 2

Troubleshooting Checklist for Odoo Servers (continued)

The practical workflow is to observe the current state, make one controlled change, and verify the result. For this lesson, that means paying attention to symptom scope, recent changes, service state, logs, ports, resources, database reachability, and rollback options and keeping the command output as evidence.

L18 · TEACHING 3

Troubleshooting Checklist for Odoo Servers (continued)

Linux rewards precision: paths, users, permissions, units, and sockets all have exact meanings. When those details are treated casually, the common failure is making multiple untracked changes and losing the ability to identify the fix, which is harder to diagnose after several unrelated changes.

L18 · TEACHING 4

Troubleshooting Checklist for Odoo Servers (continued)

In an Odoo deployment, Odoo availability depends on systemd, Python, Odoo configuration, PostgreSQL, Nginx, disk, memory, and network policy connects this topic directly to uptime, data safety, and predictable releases. The same command can be harmless on a test VM and dangerous on a production database host, so context always comes first.

Senior engineers build small repeatable checks around this area instead of relying on memory. They record the expected command, the expected output pattern, and the rollback step before changing a shared server.

L18 · TEACHING 5

Troubleshooting Checklist for Odoo Servers (continued)

The goal is not to memorize every flag, but to understand the model well enough to choose the right tool. A checklist is not bureaucracy; it is how you keep judgment available under pressure. turns Linux from a source of surprises into a stable operating base for Odoo work.

L18 · KEY POINTS

What to remember

- Start by defining the user-visible symptom and scope.
- Check recent changes before assuming random failure.
- Inspect service state, logs, ports, disk, memory, and database connectivity.
- Make one change at a time and record its result.
- Prefer rollback when a recent deployment caused the incident.
- Close the loop with prevention work, not just restoration.

L18 · EXAMPLE 1

First five minutes of an Odoo outage

First five minutes of an Odoo outage

This captures time, service state, recent Odoo logs, listening ports, disk, and memory in a repeatable order. It avoids the common mistake of restarting first and destroying useful failure context.

```
date -Is
sudo systemctl status odoo nginx postgresql --no-pager
sudo journalctl -u odoo --since "20 minutes ago" --no-pager | tail -120
ss -ltnp | grep -E ':(80|443|8069|5432)\b'
df -h
free -h
```

L18 · EXAMPLE 2

Check database reachability as Odoo

Check database reachability as Odoo

Testing from the service user's Python environment verifies more than raw network connectivity. It checks that the expected library, credentials path, and database access all work together.

```
sudo -u odoo -H bash -lc '  
    source /opt/odoo/venv/bin/activate  
    python - <<PY  
import psycopg2  
conn = psycopg2.connect(dbname="postgres", user="odoo", host="127.0.0.1")  
print("database connection ok")  
conn.close()  
PY  
'
```

L18 · PITFALLS

Common mistakes

- Restarting every service before collecting evidence.
- Changing several variables at once during an incident.
- Ignoring recent deployments, package upgrades, or certificate renewals.
- Stopping after recovery without writing the prevention task.

Remember: Validate fixes in a disposable database before production.

L18 · EXTRA

Incident order

- State the symptom and time window.
- Collect systemd, logs, ports, disk, memory, and database facts.
- Form one hypothesis and test it.
- Apply one reversible change.
- Verify user impact and document prevention.

SECTION WRAP

You should now be able to...

- L01 OS Fundamentals and Kernel Basics
- L02 Terminal Navigation with pwd, cd, ls, and tree
- L03 File Manipulation: Create, Copy, Move, Link, and Remove
- L04 Permissions with chmod, chown, and chgrp
- L05 Users, Groups, and sudo
- L06 Processes with top, htop, ps, and kill
- L07 systemd Services
- L08 journalctl and Log Files
- ... and 10 more lessons in this deck

Remember: Complete outstanding labs before starting the next section.

NEXT STEP

Complete the section labs
Then open the next presentation deck